PROJECT REPORT

Design, Simulation and Implementation Analysis of a 5-Stage Pipelined RISC-V (RV32I Subset) Processor in Verilog HDL

with Data Forwarding, Load-Use Hazard Detection and Branch Flushing

Tool flow: Xilinx Vivado (RTL simulation, synthesis, utilization and power analysis)
Course: Build a RISC-V CPU Using Verilog HDL (Infoziant)

Submitted by: gokul k
Department of Electronics and Communication Engineering
Kalasalingam Academy of Research and Education
November 2026

Abstract

This report documents the design, verification and implementation analysis of a classic five-stage pipelined processor that executes a subset of the 32-bit RISC-V (RV32I) instruction set. The processor is written in synthesizable Verilog as a single module, riscv_5stage, and is organised as Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory access (MEM) and Write Back (WB) stages separated by four pipeline registers.

Three hardware mechanisms keep the pipeline correct and busy: (1) a dynamic forwarding unit that routes results from the EX/MEM and MEM/WB registers back to the ALU inputs, (2) a load-use hazard detector that stalls the front end for one cycle and inserts a bubble, and (3) a branch resolution scheme in EX that flushes the two younger instructions when a BEQ is taken. An internal register-file bypass resolves the remaining write-back/decode conflict.

A self-checking testbench loads a nine-instruction program covering ADDI, ADD, SUB, SW, LW and BEQ. The simulation waveforms show the expected register results (x1=5, x2=7, x3=12, x4=7, x5=12, x6=0, x7=11) and memory result (MEM[0]=12), the forwarding paths firing in the expected cycles, and the taken branch skipping the instruction at address 0x1C. The report also presents the elaborated and synthesized schematics, the utilization report and the power report, and explains what each of those results does and does not tell us about the design.

1. Introduction

Pipelining is the principal technique used to raise instruction throughput in modern processors. Instead of completing one instruction before starting the next, the datapath is divided into stages and several instructions are in flight at once, each in a different stage. In an ideal pipeline one instruction completes every clock cycle once the pipeline is full, so throughput approaches one instruction per cycle (CPI of 1) while the clock period is set by the slowest single stage rather than by the whole datapath.

Overlapping instructions creates hazards: situations in which the next instruction cannot execute in its designated cycle without producing a wrong result. This design addresses all three classic types: data hazards (read-after-write dependencies), control hazards (branches) and, by using separate instruction and data memories, structural hazards on memory.

1.1 Objectives

- Implement a five-stage RISC-V pipeline for the instructions ADD, SUB, AND, OR, ADDI, LW, SW and BEQ.
- Resolve data hazards with forwarding, a register-file bypass and a one-cycle load-use stall.
- Resolve control hazards by flushing wrong-path instructions after a taken branch.
- Verify the design in simulation with a directed test program and inspect the waveforms.
- Synthesize the design in Vivado and analyse the schematic, utilization and power results.

1.2 Design Summary

Parameter	Value
Top module	riscv_5stage (ports: clk, reset, probe_pc[31:0])
ISA subset	ADD, SUB, AND, OR, ADDI, LW, SW, BEQ
Word / address width	32-bit data path, 32-bit PC
Instruction memory	64 words (reg [31:0] instruction_mem [0:63]), indexed by pc[7:2]
Data memory	64 words (reg [31:0] data_mem [0:63]), indexed by ALU result [7:2]
Register file	32 x 32-bit, x0 hard-wired to zero
Hazard handling	EX/MEM and MEM/WB forwarding, WB-to-ID bypass, load-use stall, branch flush
Branch resolution	In EX stage (2-cycle penalty when taken), static not-taken fetch
Testbench clock	100 MHz (10 ns period), reset released at 12 ns
probe_pc output	Exposes the PC so that synthesis keeps the design visible

2. Processor Architecture

Figure 1 shows the overall organisation. Each stage computes combinationally from the pipeline register in front of it and all pipeline registers are updated together on the rising clock edge. The forwarding paths run backwards from later stages to earlier ones, and branch and stall signals control the PC and the IF/ID and ID/EX registers.

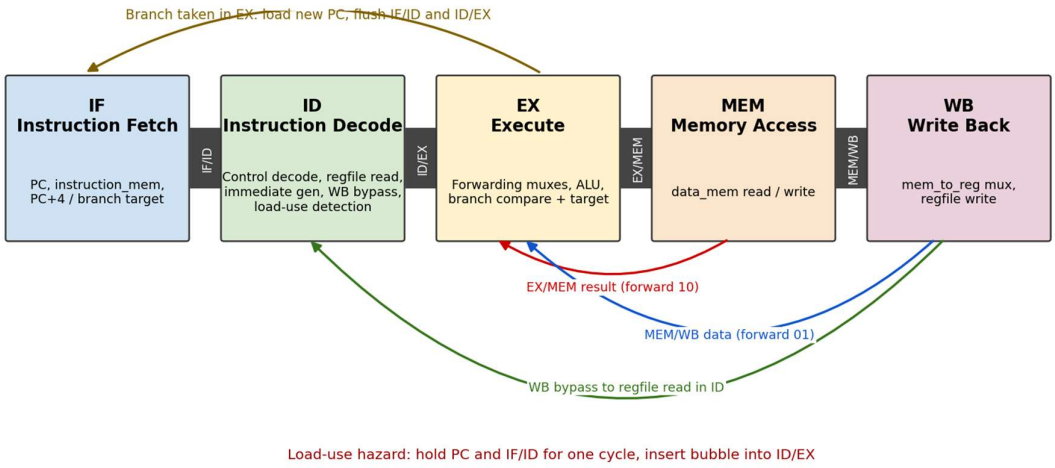


Figure 1: Five-stage pipeline organisation with forwarding, bypass, flush and stall paths

2.1 Pipeline Registers

Pipeline registers carry both data and the control signals that the later stages need. Control signals that are no longer needed are not forwarded, which keeps each register as small as possible.

Register	Data fields	Control fields
IF/ID	if_id_pc, if_id_inst	None (instruction is decoded in ID)
ID/EX	id_ex_pc, id_ex_r1, id_ex_r2, id_ex_imm, id_ex_rs1, id_ex_rs2, id_ex_rd	alu_ctrl[2:0], alu_src, mem_write, mem_read, branch, reg_write, mem_to_reg

Register	Data fields	Control fields
EX/MEM	ex_mem_alu_res, ex_mem_r2 (store data), ex_mem_rd	mem_write, mem_read, reg_write, mem_to_reg
MEM/WB	mem_wb_alu_res, mem_wb_mem_data, mem_wb_rd	reg_write, mem_to_reg

3. Pipeline Stages in Detail

3.0 How the Pipeline Works: Overview

A single instruction needs five jobs done on it: fetch it, decode it, execute it, access memory and write the result back. Instead of one hardware block doing all five jobs for one instruction before the next one starts, the processor has five dedicated stages. On every clock cycle each stage works on a different instruction, so up to five instructions are in progress at the same time. At every rising clock edge, each stage hands its result to the next stage through a pipeline register, and the instruction moves one stage forward.

Stage	Job	Register written at the clock edge	Instructions doing real work
1. IF	Read the instruction at the PC and choose the next PC	IF/ID (and pc)	All
2. ID	Decode, read registers, build the immediate, detect load-use hazards	ID/EX	All
3. EX	Forward operands, run the ALU, compare for branches	EX/MEM	All (ALU, address, branch compare)
4. MEM	Read or write the data memory	MEM/WB	LW and SW only
5. WB	Pick ALU result or loaded data and write it to the register file	registers[rd]	ADD, SUB, AND, OR, ADDI, LW

Snapshot of the pipeline in cycle 7 of the test program (five different instructions in flight at once):

IF	ID	EX	MEM	WB
beq x4,x2 (0x18)	lw x5,0(x0) (0x14)	sw x3,0(x0) (0x10)	sub x4,x3,x1 (0x0C)	add x3,x1,x2 (0x08)

Snapshot in cycle 9, when the branch is resolved:

IF	ID	EX	MEM	WB
fetch at 0x20 (will be discarded)	addi x6,x0,99 (wrong path, will be squashed)	beq x4,x2: TAKEN	lw x5,0(x0)	sw x3,0(x0)

Below, each stage is described with its inputs, what it does, and its outputs, using the signal names from the Verilog source.

3.1 Stage 1: Instruction Fetch (IF)

Inputs	Processing	Outputs
pc, instruction_mem, branch_taken, branch_target, load_use_hazard	Read instruction_mem[pc[7:2]]; compute pc + 4; select PC+4, branch target, or hold the PC	if_id_inst, if_id_pc, updated pc

The IF stage reads the instruction at the address held in the program counter and prepares the next address. The instruction memory is word-addressed through pc[7:2], so the two least significant PC bits are never used for indexing.

```
if (branch_taken) begin
    pc <= branch_target;
    if_id_inst <= 32'h00000013;   // flush with NOP
    if_id_pc   <= branch_target;
end else if (!load_use_hazard) begin
    if_id_inst <= instruction_mem[pc[7:2]];
    if_id_pc   <= pc;
    pc         <= pc + 32'd4;
end
```

- **Normal operation:** the instruction is latched into IF/ID, the PC value is saved alongside it (needed later for branch target calculation) and the PC advances by 4.
- **Stall:** when load_use_hazard is high neither the PC nor IF/ID is updated, so the same instruction is presented again next cycle.
- **Taken branch:** the PC is loaded with branch_target and IF/ID receives a NOP (ADDI x0,x0,0, encoded 0x00000013), discarding the wrong-path instruction fetched this cycle.

3.2 Stage 2: Instruction Decode (ID)

Inputs	Processing	Outputs
if_id_inst, if_id_pc, register file, wb_write_data, mem_wb_rd, id_ex_mem_read, id_ex_rd	Split fields, decode control, read rs1/rs2 with WB bypass, generate immediate, check load-use hazard	id_ex_pc, r1, r2, imm, rs1, rs2, rd and the 7 control bits (or a bubble)

The instruction in IF/ID is split into its fields (opcode, rd, funct3, rs1, rs2, funct7). Three things happen in parallel: the control unit decodes the opcode, the register file is read, and the immediate is generated.

- **Register read with bypass:** if the instruction currently in WB is writing the register being read (mem_wb_reg_write and mem_wb_rd equal to rs1 or rs2), the new value wb_write_data is taken directly. This closes the three-cycle gap in which the register file would still hold the stale value. Reads of x0 always return zero.
- **Immediate generation:** I-type imm_i = sign-extended inst[31:20]; S-type imm_s = {inst[31:25], inst[11:7]}; B-type imm_b = {inst[31], inst[7], inst[30:25], inst[11:8], 1'b0}, all sign-extended to 32 bits.
- **Control decode:** a combinational case statement on the opcode produces the eight control signals listed in Section 5. Unrecognised opcodes leave every control signal at zero, which makes them harmless bubbles.

- **Hazard detection:** load_use_hazard = id_ex_mem_read && id_ex_rd != 0 && (id_ex_rd == rs1 || id_ex_rd == rs2). When true, a bubble (all write/branch controls cleared) is written into ID/EX instead of the decoded instruction.

3.3 Stage 3: Execute (EX)

Inputs	Processing	Outputs
id_ex_* registers, ex_mem_alu_res, wb_write_data	Choose forwarded operands, run the ALU, compare operands for BEQ, add PC + immediate for the target	ex_mem_alu_res, ex_mem_r2, ex_mem_rd, controls; branch_taken and branch_target back to IF

The EX stage performs the arithmetic and resolves branches. Before the ALU, two forwarding multiplexers select the freshest value of each source operand.

forward select	Meaning	Source of operand
2'b10	Producer is the instruction one ahead (now in EX/MEM)	ex_mem_alu_res
2'b01	Producer is two ahead (now in MEM/WB)	wb_write_data (ALU result or loaded data)
2'b00	No dependency in flight	id_ex_r1 / id_ex_r2 read in ID

EX/MEM has priority over MEM/WB so that the most recent producer wins when both match, and x0 is excluded from forwarding. The second operand is then chosen between the forwarded register value (fwd_in2) and the immediate according to alu_src. Using fwd_in2 as the store-data source means that SW also receives forwarded data.

alu_ctrl	Operation	Used by
000	alu_in1 + alu_in2	ADD, ADDI, LW and SW (address calculation)
001	alu_in1 - alu_in2	SUB and BEQ (comparison)
010	alu_in1 & alu_in2	AND
011	alu_in1 | alu_in2	OR

Branch decision: branch_taken = id_ex_branch && (alu_in1 == fwd_in2), and the target is branch_target = id_ex_pc + id_ex_imm. The comparator operates on forwarded values, so a BEQ that depends on an immediately preceding ALU instruction still compares the correct data. A taken branch bubbles ID/EX and flushes IF/ID and the PC load, so two wrong-path slots are cancelled: a 2-cycle branch penalty.

3.4 Stage 4: Memory Access (MEM)

Inputs	Processing	Outputs
ex_mem_alu_res (address), ex_mem_r2 (store data), mem_read, mem_write	Read data_mem for LW; write data_mem for SW; pass everything else through	mem_wb_alu_res, mem_wb_mem_data, mem_wb_rd, reg_write, mem_to_reg

Only LW and SW do real work here. The ALU result from EX/MEM is the byte address; bits [7:2] select one of 64 words.

- **Load:** if ex_mem_mem_read, data_mem[ex_mem_alu_res[7:2]] is captured into mem_wb_mem_data; otherwise zero is captured.
- **Store:** if ex_mem_mem_write, ex_mem_r2 (the possibly forwarded rs2 value) is written to data_mem.
- **All other instructions:** the ALU result, destination register and control bits pass through unchanged into MEM/WB.

3.5 Stage 5: Write Back (WB)

Inputs	Processing	Outputs
mem_wb_alu_res, mem_wb_mem_data, mem_to_reg, reg_write, mem_wb_rd	Select loaded data or ALU result; write it to registers[rd] unless rd is x0	registers[rd]; wb_write_data for forwarding and bypass

The WB stage selects between the loaded data and the ALU result with wb_write_data = mem_wb_mem_to_reg ? mem_wb_mem_data : mem_wb_alu_res and, when mem_wb_reg_write is set and mem_wb_rd is not zero, writes it into the register file. Register x0 is re-forced to zero every cycle. The same wb_write_data wire feeds the MEM/WB forwarding path and the ID-stage bypass.

3.6 Worked Example: add x3, x1, x2 Moving Through the Pipeline

This traces the instruction at address 0x08 (machine code 0x002081B3) cycle by cycle, with the actual values from the test program. The two instructions before it (addi x1,x0,5 and addi x2,x0,7) are still in the pipeline when it is fetched, which is why forwarding is needed.

Cycle	Stage	What happens	Values
C3	IF	instruction_mem[2] is read and latched in IF/ID; PC becomes 0x0C	if_id_inst = 0x002081B3
C4	ID	Fields split: opcode 0110011 (R-type), rd=3, rs1=1, rs2=2, funct7=0000000. Control: reg_write=1, alu_ctrl=000. The register file is read, but x1 and x2 are not written yet, so the values read are stale	id_ex_r1 = 0, id_ex_r2 = 0 (stale)
C5	EX	Forwarding unit overrides the stale values: x1 comes from MEM/WB (forward_a = 01) and x2 from EX/MEM (forward_b = 10). The ALU adds them	alu_in1 = 5, alu_in2 = 7, alu_result = 12
C6	MEM	No memory access (mem_read = mem_write = 0). The result moves on to MEM/WB. In this same cycle the SUB behind it forwards this 12	mem_wb_alu_res = 12
C7	WB	mem_to_reg = 0 selects the ALU result; at the clock edge it is written into the register file. In this same cycle the SW behind it receives 12 by forwarding	registers[3] = 12

The same five-step pattern applies to every instruction. What differs is which control bits are set in ID, and therefore which stages do real work: SW writes memory in MEM and writes no register in WB, LW reads memory in MEM and selects the loaded data in WB, and BEQ writes nothing but redirects the PC from EX.

4. Hazard Handling

Hazard	Example from the test program	Mechanism	Cost
RAW, producer 1 ahead	ADD x3 needs x2 from ADDI x2 (one instruction earlier)	Forward EX/MEM to ALU (forward_b = 10)	0 cycles
RAW, producer 2 ahead	ADD x3 needs x1 from ADDI x1; SW needs x3 from ADD	Forward MEM/WB to ALU (forward = 01)	0 cycles
RAW, producer 3 ahead	Instruction in ID reads a register being written in WB	Register-file bypass in ID	0 cycles
Load-use	A dependent instruction directly after LW (not triggered in the test)	Stall PC and IF/ID one cycle, bubble in ID/EX, then forward	1 cycle
Control	BEQ x4,x2 taken to 0x20	Resolve in EX, load PC, flush IF/ID and ID/EX	2 cycles

A load result is not available until the end of MEM, so even full forwarding cannot supply it to the instruction directly behind the load. That is why a one-cycle stall is unavoidable, and why the stall is implemented separately from the forwarding unit.

5. Instruction Set and Per-Instruction Operation

5.1 Supported Instructions and Control Signals

Instr.	Type	Opcode	funct3	funct7	Operation	Reg write	ALU src	Mem R	Mem W	Mem to reg	Branch	ALU ctrl
ADD	R	0110011	000	0000000	rd = rs1 + rs2	1	0	0	0	0	0	000
SUB	R	0110011	000	0100000	rd = rs1 - rs2	1	0	0	0	0	0	001
AND	R	0110011	111	-	rd = rs1 & rs2	1	0	0	0	0	0	010
OR	R	0110011	110	-	rd = rs1 | rs2	1	0	0	0	0	0	011
ADDI	I	0010011	-	-	rd = rs1 + imm	1	1	0	0	0	0	000
LW	I	0000011	-	-	rd = MEM[rs1+imm]	1	1	1	0	1	0	000
SW	S	0100011	-	-	MEM[rs1+imm] = rs2	0	1	0	1	0	0	000
BEQ	B	1100011	-	-	if rs1==rs2: PC += imm	0	0	0	0	0	1	001

Note: the decoder tests funct3 only for the R-type group and does not check funct7 for AND and OR, and it does not check funct3 for ADDI, LW, SW and BEQ. This is acceptable for the supported subset but means that other encodings sharing these opcodes would be mis-decoded (see Section 9).

5.2 How Each Instruction Flows Through the Pipeline

ADD rd, rs1, rs2 (R-type)

- IF: instruction fetched, PC advances by 4.
- ID: opcode 0110011 sets reg_write; funct3=000 with funct7=0000000 selects alu_ctrl 000; rs1 and rs2 are read (with WB bypass).
- EX: forwarding muxes pick the freshest rs1/rs2; ALU adds them; alu_src=0 keeps the register operand.
- MEM: no memory access; result passes to MEM/WB.
- WB: mem_to_reg=0 selects the ALU result, which is written to rd.

Test program: add x3, x1, x2 (0x002081B3) produces 5 + 7 = 12 in x3. x1 arrives by MEM/WB forwarding and x2 by EX/MEM forwarding.

SUB rd, rs1, rs2 (R-type)

Identical to ADD except that funct7=0100000 selects alu_ctrl 001, so the ALU computes rs1 - rs2.

Test program: sub x4, x3, x1 (0x40118233) gives 12 - 5 = 7. x3 is forwarded from EX/MEM (forward_a = 10).

AND and OR (R-type)

Same flow as ADD with funct3=111 (alu_ctrl 010, bitwise AND) or funct3=110 (alu_ctrl 011, bitwise OR). They are implemented but not exercised by the supplied testbench.

ADDI rd, rs1, imm (I-type)

- ID: imm_i is generated and alu_src=1 so the immediate replaces the second operand.
- EX: alu_in1 + imm. MEM: pass-through. WB: ALU result written to rd.

Test program: addi x1, x0, 5 (0x00500093), addi x2, x0, 7 (0x00700113) and, after the branch, addi x7, x0, 11 (0x00B00393). ADDI x0,x0,0 is also the NOP used for flushing and for initial memory contents.

SW rs2, imm(rs1) (S-type)

- ID: imm_s is built from two instruction fields; mem_write=1, alu_src=1, reg_write=0.
- EX: ALU computes the address rs1 + imm; the store data is fwd_in2 so it can be forwarded.
- MEM: data_mem[address[7:2]] <= ex_mem_r2. WB: nothing is written.

Test program: sw x3, 0(x0) (0x00302023) stores 12 at address 0. The store data is forwarded from MEM/WB (forward_b = 01).

LW rd, imm(rs1) (I-type)

- ID: mem_read=1, mem_to_reg=1, reg_write=1, alu_src=1, imm_i.
- EX: address = rs1 + imm. MEM: the word is read into mem_wb_mem_data.
- WB: mem_to_reg=1 selects the loaded data, written to rd. A dependent instruction directly behind LW would trigger the load-use stall.

Test program: lw x5, 0(x0) (0x00002283) reads back the 12 that SW stored, so x5 = 12. Because the next instruction (BEQ) uses x4 and x2 only, no stall occurs.

BEQ rs1, rs2, offset (B-type)

- ID: branch=1, alu_ctrl=001, imm_b generated; no register write.
- EX: branch_taken when the forwarded operands are equal; branch_target = PC of the BEQ + imm.
- If taken: PC loads the target, IF/ID is replaced by a NOP and ID/EX receives a bubble. If not taken: the pipeline continues unchanged.

Test program: beq x4, x2, +8 at address 0x18 (0x00220463). x4 = x2 = 7, so the branch is taken to 0x18 + 8 = 0x20 and the instruction at 0x1C (addi x6, x0, 99) never reaches EX, leaving x6 = 0.

6. Verification Methodology

The testbench tb_riscv_5stage instantiates the design, generates a 100 MHz clock, holds reset for 12 ns, and then runs 25 clock cycles so that the pipeline drains. Instruction encoder functions (enc_i, enc_r, enc_s, enc_b) build the machine code, and the instruction and data memories are written directly through hierarchical references (dut.instruction_mem, dut.data_mem). All unused instruction slots are filled with NOPs.

6.1 Test Program

Addr	Index	Assembly	Machine code	Expected effect
0x00	0	addi x1, x0, 5	`0x00500093`	x1 = 5
0x04	1	addi x2, x0, 7	`0x00700113`	x2 = 7
0x08	2	add x3, x1, x2	`0x002081B3`	x3 = 12 (forwarding from both MEM/WB and EX/MEM)
0x0C	3	sub x4, x3, x1	`0x40118233`	x4 = 7 (forward from EX/MEM)
0x10	4	sw x3, 0(x0)	`0x00302023`	MEM[0] = 12 (store data forwarded from MEM/WB)
0x14	5	lw x5, 0(x0)	`0x00002283`	x5 = 12
0x18	6	beq x4, x2, +8	`0x00220463`	Taken (7 = 7), target 0x20
0x1C	7	addi x6, x0, 99	`0x06300313`	Skipped by branch, x6 stays 0
0x20	8	addi x7, x0, 11	`0x00B00393`	x7 = 11 (branch target)

The machine codes above match the instruction values visible in the instruction memory array of the simulation waveform (00500093, 00700113, 002081b3, 40118233, 00302023, 00002283, 00220463, 06300313, 00b00393).

6.2 Expected Pipeline Schedule

Cycle k ends at the rising clock edge at roughly 10k + 5 ns in the simulation (cycle 1 ends at the first edge after reset is released). The chart shows which stage each instruction occupies. FWD marks cycles in which forwarding is active in EX, and X marks the squashed instruction.

Instr.	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
I0 addi x1	IF	ID	EX	MEM	WB									
I1 addi x2		IF	ID	EX	MEM	WB								
I2 add x3			IF	ID	EX	MEM	WB							
I3 sub x4				IF	ID	EX	MEM	WB						
I4 sw x3					IF	ID	EX	MEM	WB					
I5 lw x5						IF	ID	EX	MEM	WB				
I6 beq							IF	ID	EX	MEM	WB			
I7 addi x6								IF	**ID X**					
I8 addi x7										IF	ID	EX	MEM	WB

- **C5:** ADD is in EX while ADDI x1 is in WB (forward_a = 01) and ADDI x2 is in MEM (forward_b = 10).

- **C6:** SUB is in EX; ADD in MEM supplies x3 (forward_a = 10).
- **C7:** SW is in EX; ADD in WB supplies the store data (forward_b = 01).
- **C9:** BEQ is in EX and branch_taken goes high. addi x6 (in ID) is converted to a bubble and the fetch in progress is replaced by a NOP, so the PC reloads 0x20.
- **C10 to C14:** addi x7 is fetched from the branch target and completes in C14, which is why the simulation needs several drain cycles before the final check.

6.3 Stage Occupancy per Clock Cycle

The table below lists which instruction occupies each stage in every cycle. Bubble and NOP entries after C9 show the cost of the taken branch.

Cycle	IF	ID	EX	MEM	WB
C1	I0	-	-	-	-
C2	I1	I0	-	-	-
C3	I2	I1	I0	-	-
C4	I3	I2	I1	I0	-
C5	I4	I3	I2	I1	I0
C6	I5	I4	I3	I2	I1
C7	I6	I5	I4	I3	I2
C8	I7	I6	I5	I4	I3
C9	discarded	I7 (squashed)	I6	I5	I4
C10	I8	NOP (flush)	bubble	I6	I5
C11	NOP	I8	NOP	bubble	I6
C12	NOP	NOP	I8	NOP	bubble
C13	NOP	NOP	NOP	I8	NOP
C14	NOP	NOP	NOP	NOP	I8

7. Simulation Results and Waveform Analysis

7.1 Control, Forwarding and ALU Signals

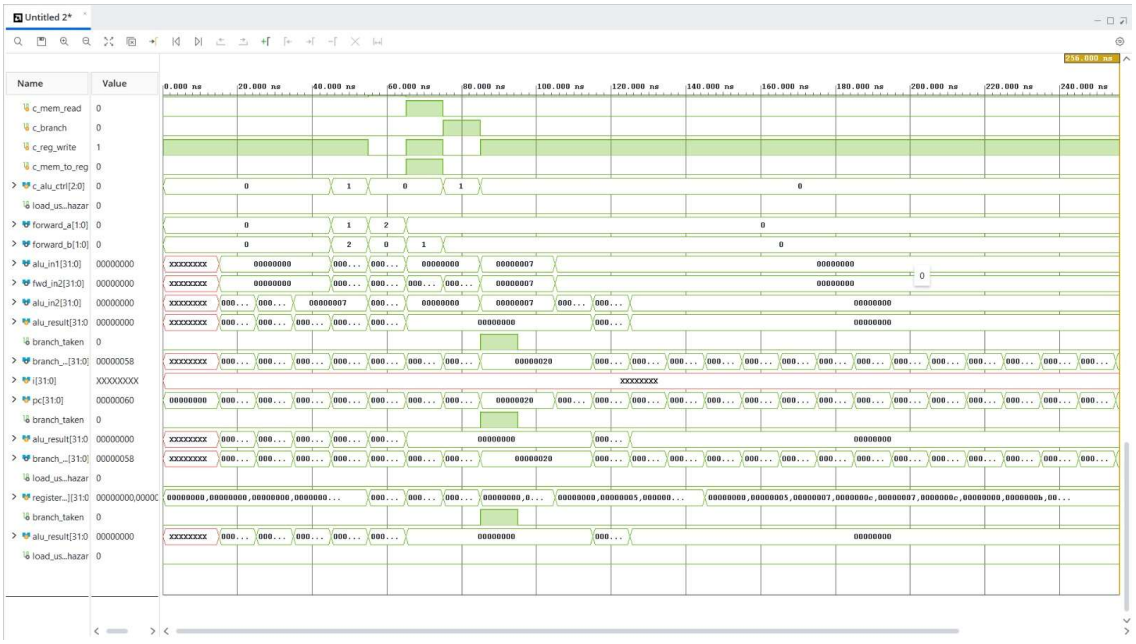


Figure 2: Simulation waveform of control signals, forwarding selects, ALU signals, branch and register file

Figure 2 shows the following behaviour, which agrees with the schedule in Section 6.2:

- **c_alu_ctrl** is 0 for the ADDI/ADD group, becomes 1 for SUB, returns to 0 for SW and LW (address addition) and is 1 again for BEQ (subtract-to-compare).
- **c_reg_write** drops low for SW and for BEQ, the two instructions that do not write a register. c_mem_read and c_mem_to_reg pulse only for LW, and c_branch pulses only for BEQ.
- **forward_a / forward_b** show the sequence (a=1, b=2) for ADD, (a=2, b=0) for SUB and (a=0, b=1) for SW, exactly as predicted. They remain 0 for the rest of the run.
- **load_use_hazard** stays at 0 for the entire run, confirming that this program contains no load-use dependency.
- **branch_taken** produces a single pulse near 85 to 95 ns, at which point branch_target is 0x00000020 and alu_in1 = fwd_in2 = 7 (the two values compared by BEQ).
- **register array** grows step by step and settles at 00000000, 00000005, 00000007, 0000000c, 00000007, 0000000c, 00000000, 0000000b for x0 to x7, matching the expected results with x6 correctly left at zero.

The row labelled i[31:0] shows XXXXXXXX because it is the integer loop variable used for register reset and it is not meaningful for functional checking.

7.2 Pipeline Register and Memory Contents

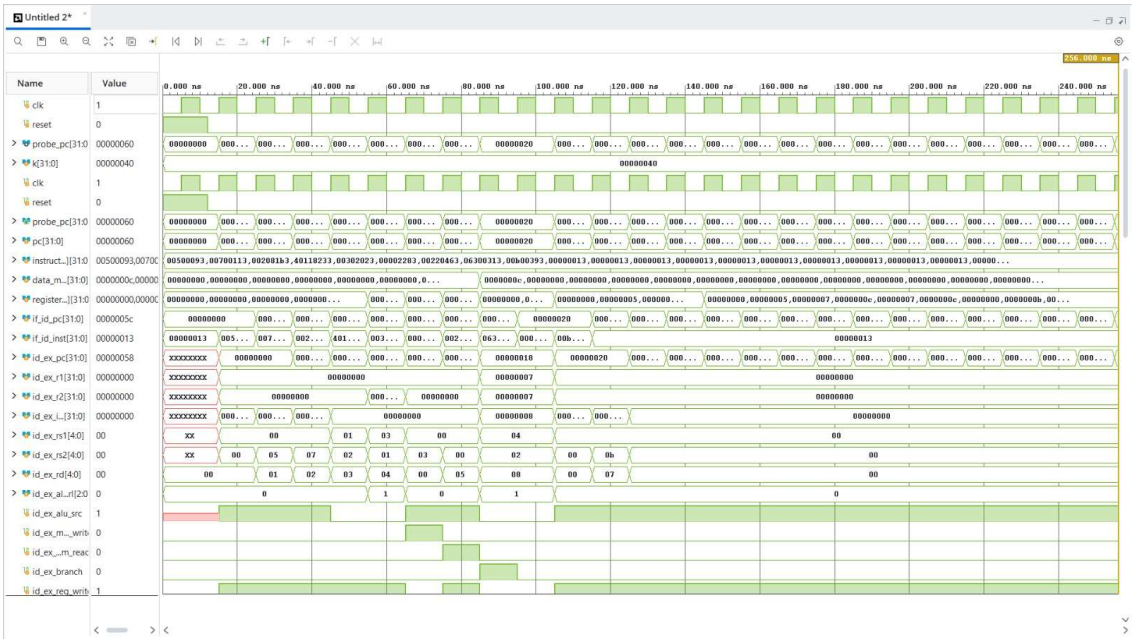


Figure 3: Simulation waveform of clock, reset, PC, instruction memory, data memory, register file and ID/EX pipeline register

- **Reset and clock:** reset is high for the first 12 ns; the clock period is 10 ns.
- **PC:** probe_pc and pc increment by 4 each cycle, jump to 0x20 after the branch resolves, and then continue sequentially through the NOP-filled memory. The final value in the capture is 0x60 at 256 ns.
- **Data memory:** data_mem[0] changes from 0 to 0000000c when the store reaches MEM, confirming the SW operation.
- **ID/EX register:** id_ex_rd follows 01, 02, 03, 04, 00 (SW), 05, 08, 00 (bubble), 07. The value 08 on the BEQ is not a real destination: for B-type instructions bits [11:7] hold part of the immediate. The 00 slot after it is the bubble inserted by the branch flush, followed by 07 for addi x7.
- **id_ex_rs2** shows 05, 07 and 0b for the three ADDI instructions because, for I-type instructions, the rs2 bit-field holds the low bits of the immediate (5, 7 and 11). This is harmless for execution (alu_src selects the immediate), but see the load-use note in Section 9.
- **id_ex_pc** shows 00000018 for the BEQ, from which the branch target 0x20 is calculated using the immediate 8 (id_ex_imm).

8. Schematic, Utilization and Power Analysis

8.1 Elaborated (RTL) Schematics

The elaborated schematics show the design as Vivado understands it directly from the RTL, before technology mapping. They contain the full pipeline: registers, multiplexers, comparators, adders and control logic.

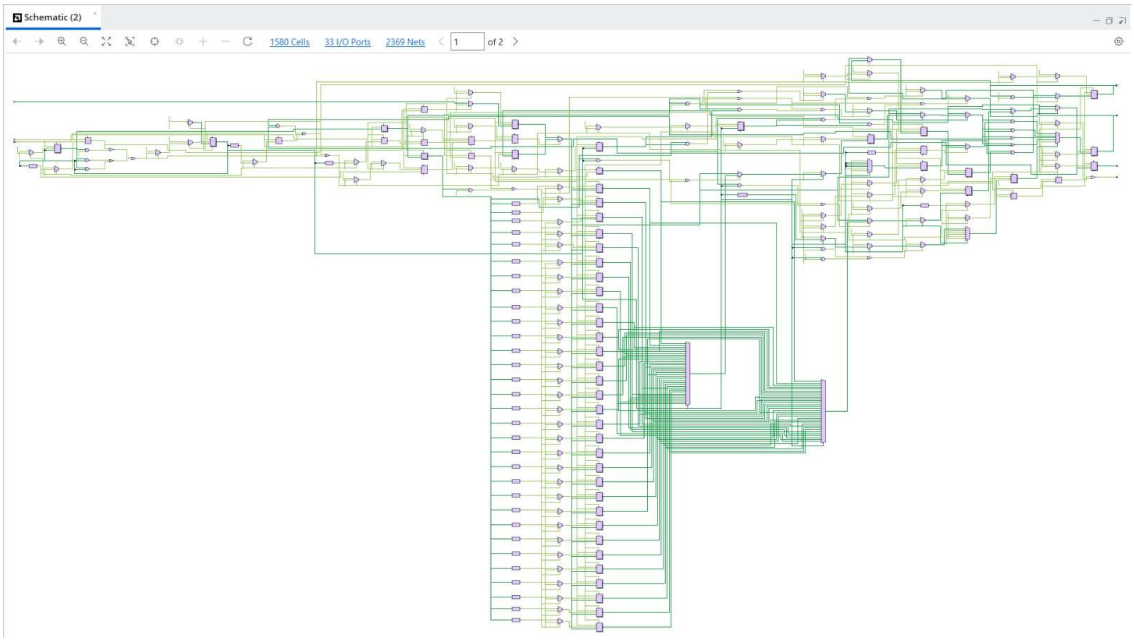


Figure 4: Schematic view, page 1 of 2 (1580 cells, 33 I/O ports, 2369 nets)

Figure 4 shows the datapath at gate and block level. The tall column of registers and multiplexers corresponds to the 32-entry register file with its read-port selection, and the dense bundle of wires near the lower right is the 32-bit data buses feeding the wide read multiplexers. The upper right region contains the hazard, forwarding and branch logic, and the upper left region holds the PC and instruction decode logic.

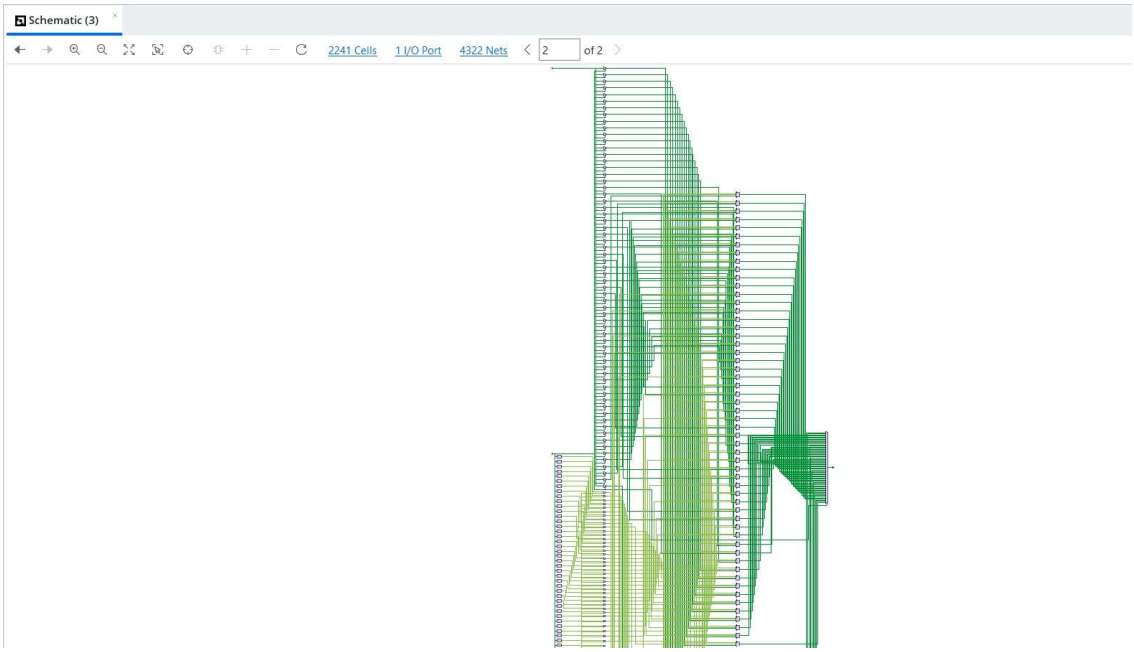


Figure 5: Schematic view, page 2 of 2 (2241 cells, 1 I/O port, 4322 nets)

Figure 5 is the second schematic view of the larger netlist (2241 cells, 4322 nets, 1 I/O port). It is dominated by many parallel 32-bit signal groups, which is typical of register-file and memory arrays being drawn bit by bit.

8.2 Synthesized Schematic

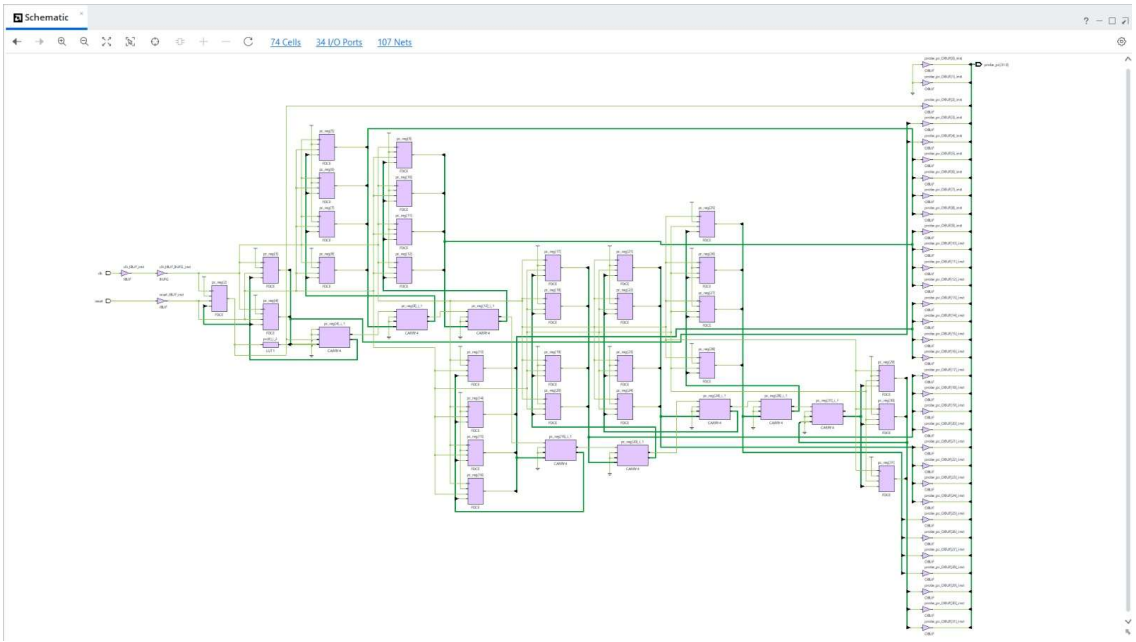


Figure 6: Post-synthesis schematic (74 cells, 34 I/O ports, 107 nets)

After synthesis the schematic shrinks to 74 cells: an IBUF and BUFG on clk, an IBUF on reset, 30 FDCE flip-flops (pc_reg[2] to pc_reg[31]), eight CARRY4 cells forming the incrementer, one LUT, and 32 OBUFs driving probe_pc[31:0]. This is a 32-bit program counter and nothing else. Section 8.3 explains why.

8.3 Utilization and Interpretation

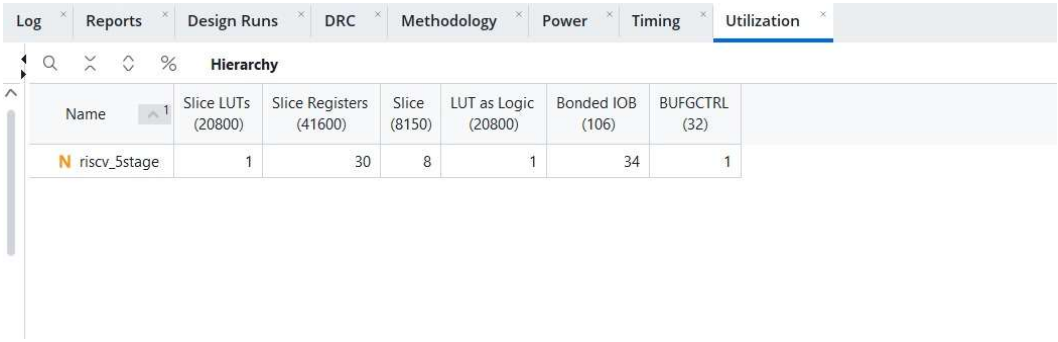


Figure 7: Vivado utilization report (hierarchy view)

Resource	Used	Available	Utilization
Slice LUTs (LUT as Logic)	1	20,800	under 0.01%
Slice registers	30	41,600	0.07%
Slices	8	8,150	0.10%
Bonded IOB	34	106	32.1%
BUFGCTRL	1	32	3.1%

The device totals (20,800 LUTs, 41,600 registers, 106 bonded IOBs) correspond to an Artix-7 class FPGA of the XC7A35T size.

The utilization figures describe a 32-bit counter, not the full processor. The reasons are visible in the RTL:

- instruction_mem has no initial contents in the synthesizable source. The program is loaded only by the testbench through hierarchical references, which synthesis cannot see.
- Synthesis therefore treats the instruction memory as never written, so every decoded instruction is a constant, the control signals collapse, and the register file, ALU, forwarding logic and data memory have no observable effect on any output.
- The only logic that still reaches an output port is the PC, which drives probe_pc. The PC increments by 4, so bits [1:0] are always zero: 30 flip-flops remain, matching the 30 slice registers reported.
- The (* dont_touch = "true" *) attribute on the module stops Vivado from flagging the design as empty, but it does not prevent optimisation of the logic inside.

The elaborated schematic (1580 cells) is therefore the accurate picture of the processor structure, while the post-synthesis utilization and power numbers reflect the reduced design. To obtain real resource figures for the full CPU, the instruction memory must be initialised inside the RTL (an initial block or $readmemh of a program file) so that the program is part of the synthesized design, and the data memory result must be made observable (for example through an output port).

8.4 Power Analysis

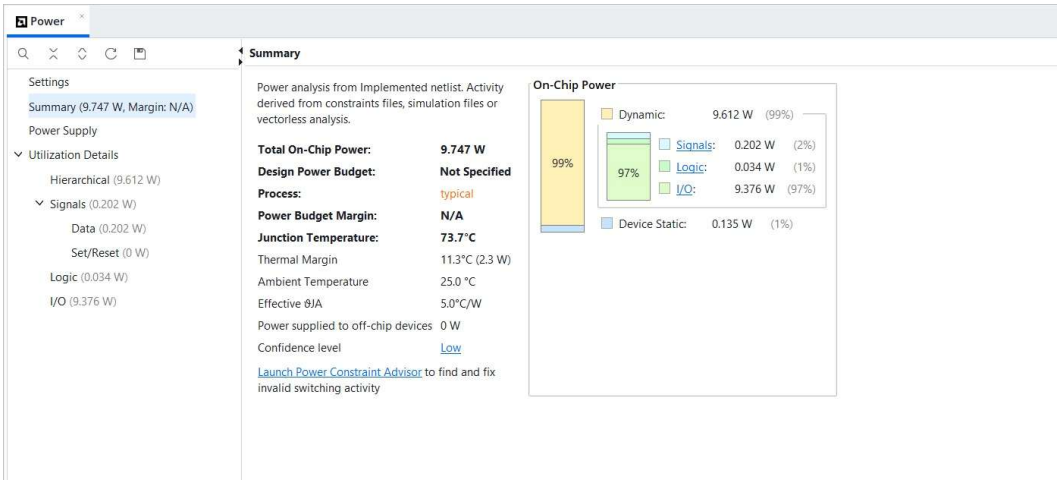


Figure 8: Vivado power report summary

Quantity	Value
Total on-chip power	9.747 W
Dynamic power	9.612 W (99%)
Signals	0.202 W (2%)
Logic	0.034 W (1%)
I/O	9.376 W (97%)
Device static power	0.135 W (1%)
Junction temperature	73.7 C
Thermal margin	11.3 C (2.3 W)

Quantity	Value
Ambient temperature / effective theta-JA	25.0 C / 5.0 C/W
Confidence level	Low

The report is dominated by I/O power (97%) because the 32 probe_pc outputs are unconstrained and the tool assumes default I/O standard, loading and switching activity. Signal and logic power together are only 0.236 W. Vivado itself rates the confidence as Low, since the activity comes from vectorless estimation rather than a simulation activity file (SAIF) and no pin constraints are present. The 9.7 W total should not be quoted as the power of this processor. It is a default-assumption figure for an unconstrained 32-bit counter with 32 output pins, and a real board-level design would not drive a 32-bit probe bus off-chip.

A more meaningful number would be obtained by (1) removing or constraining the probe outputs (for example by registering a single reduction bit), (2) running post-implementation timing simulation and importing the SAIF file, and (3) re-running power analysis on the full design after the memory-initialisation fix above.

9. Limitations and Future Work

- **Load-use detection is conservative.** It compares id_ex_rd against the rs1 and rs2 bit-fields of every instruction, including I-type instructions where the rs2 field is really immediate bits. A false one-cycle stall could occur, for example when an ADDI immediate happens to equal the destination of a preceding load. It costs performance but not correctness.
- **Incomplete decode.** funct7 is not checked for AND and OR, and funct3 is not checked for I-, S- and B-type opcodes, so unsupported variants (such as BNE or SLTI) would be decoded as BEQ or ADDI rather than rejected.
- **Instruction coverage.** AND, OR, a not-taken branch and the load-use stall path are not exercised by the supplied test program. Adding these cases would raise confidence.
- **Branch performance.** Branches resolve in EX with a 2-cycle penalty. Moving resolution to ID or adding a branch predictor would reduce it.
- **Missing instructions.** JAL, JALR, LUI, AUIPC, shifts, SLT, XOR and sub-word loads and stores are not implemented.
- **Memory model.** Memories are small behavioural arrays with no alignment check and with the unused upper address bits ignored.
- **Reset coverage.** Several pipeline-register data fields (for example mem_wb_alu_res and mem_wb_mem_data) are not reset, which is harmless functionally because their control bits are cleared, but they appear as X at the start of the simulation.
- **Implementation flow.** Add pin constraints, a timing constraint and a program-loaded ROM, then report WNS, utilization and power for the complete processor.

10. Conclusion

A five-stage pipelined RISC-V processor supporting ADD, SUB, AND, OR, ADDI, LW, SW and BEQ was designed in Verilog and verified with a directed test program. The simulation shows correct operation of every pipeline stage, correct forwarding in the three dependency cases of the test, correct memory write and read-back, and a correctly resolved taken branch that skips the intended instruction. Final register and memory values match the expected results.

The synthesis, utilization and power results were analysed critically. Because the program memory is loaded only by the testbench, the synthesized netlist retains only the program counter, so those figures are not representative of the full processor. The report identifies the cause and the changes needed to obtain meaningful implementation numbers, which form the natural next step for this project.